STUDY

AI Coding Productivity: Measured vs. Claimed

An analysis of AI coding assistant productivity, comparing real-world effectiveness with marketing claims supported by comprehensive empirical data.

August 2026

Caspr.

STUDY REPORT

AI Coding Productivity: Measured vs. Claimed

Executive Summary

Finding 1 — Controlled studies show the strongest productivity gains

As of August 2026, the strongest evidence for AI coding assistants comes from controlled experiments, which generally report significant productivity gains. However, the magnitude varies materially by study design, developer proficiency, task type, and work environment, with gains in controlled settings typically exceeding those seen in open-source or enterprise contexts.

Finding 2 — Vendor productivity claims are indicative, not conclusive

Figures from vendors such as GitHub Copilot and OpenAI are constrained by narrow task definitions and self-reported time savings. The report advises treating these claims as directional signals rather than credible proof, and recommends relying on independent empirical studies for realistic productivity assessment.

Finding 3 — Enterprise impact is positive but limited and uneven

Evidence from real-world settings, including Google's enterprise randomized controlled trial, shows notable but limited speed improvements with significant variability across developers and contexts. This suggests AI assistants can help in production environments, but their impact is less consistent and generally smaller than headline experimental results imply.

Finding 4 — More engineering activity does not reliably convert into shipped output

Quasi-experiments and large-scale observational studies indicate that AI can increase visible engineering activity, such as commit volume, without proportionate gains in delivered software. One cited example found only a 30% increase in software releases, with human review, integration, and judgment remaining key bottlenecks.

Finding 5 — Perceived productivity often exceeds measured outcomes

Developers frequently report larger productivity benefits than objective measures show. The report attributes this gap in part to increased verification effort and reduced focused work time, indicating that subjective satisfaction with AI tools does not necessarily translate into real output gains.

Finding 6 — Benefits are concentrated in routine work and less-experienced developers

AI coding assistants appear to deliver greater value for junior developers and for routine, well-scoped tasks. Benefits diminish for senior developers and for complex work requiring deep expertise, architectural judgment, and nuanced decision-making; the report also warns that heavy reliance may create learning trade-offs and potential skill degradation over time.

1

Introduction and Research Context

1

Introduction and Research Context

AI coding assistants have moved from niche autocomplete tools to mainstream development infrastructure, but the evidence base on productivity remains materially more mixed than vendor messaging suggests. Across the current literature available as of August 25, 2026, the strongest conclusion is not that these tools uniformly deliver a single productivity uplift, but that measured effects vary sharply by study design, developer experience, task type, and work context, with controlled experiments often showing larger gains than open source or enterprise field settings.[1] [10] [2]

This report therefore treats vendor claims as claims, not as evidence, and anchors interpretation in independently measured empirical studies wherever possible. The research context is defined by three recurring tensions in the literature: perceived versus measured productivity, short-term throughput versus downstream software outcomes, and immediate assistance versus longer-term skill formation and developer experience.[3] [33] [14]

1.1

The rise of AI coding assistants and the gap between marketing claims and independent evidence

AI coding assistants have scaled rapidly because they promise immediate, visible help on common development tasks such as code completion, boilerplate generation, refactoring suggestions, and conversational problem solving. That promise has been reinforced by prominent vendor figures such as GitHub Copilot's widely cited 55% faster claim and later internal reports from major model providers describing time savings that can reach 80% in selected tasks or benchmarked workflows, but those figures generally come from vendor-designed studies, self-reported outcomes, or tightly scoped internal evaluations rather than broad independent field evidence.[1] [10] [33]

Independent evidence paints a more uneven picture.

- In Google's randomized controlled trial with 96 engineers, the best estimate was about a 21% reduction in time on task, but the authors explicitly note wide confidence intervals and caution against generalizing the lab result across tools, organizations, or time periods.[10]
- In the METR randomized controlled trial on experienced open source developers, participants predicted a 24% speedup and later estimated a 20% speedup, yet measured completion time was 19% slower with AI allowed across 246 tasks, making it one of the clearest demonstrations of a perception-reality gap in this literature.[2]
- In the May 2026 meta-analysis of 23 studies and 27 effect sizes, the pooled productivity effect was positive but moderate at Hedges' g = 0.33 with 95% CI [0.09, 0.58], alongside substantial heterogeneity, meaning the average effect masks large variation across settings and populations.[1]

The practical implication is that marketing claims usually describe what is possible under favorable conditions, while independent studies ask what is typical, reproducible, and durable. For decision makers, that distinction matters because the current evidence suggests AI assistance often increases visible coding activity and perceived speed, but does not yet

support a universal claim of large productivity gains across all teams, tasks, or seniority levels.[3] [14]

1.2

Overview of the study landscape: RCTs, quasi-experiments, meta-analyses, and observational studies

The study landscape is now broad enough to support comparative interpretation, but it remains methodologically fragmented. As of 2026, the literature includes randomized controlled trials, quasi-experiments in enterprise settings, meta-analyses, systematic reviews, longitudinal mixed-methods studies, and large-scale observational analyses, each answering a different question about productivity rather than measuring a single universal construct.[1] [14] [4]

The main study types contribute different forms of evidence.

- Randomized controlled trials: These provide the strongest causal evidence on short-run task performance. Examples include Google's 96-engineer enterprise RCT and the METR open source RCT, which reached materially different conclusions despite both using experimental control, underscoring the importance of context and participant profile.[10] [2]
- Quasi-experiments: These are useful when randomization is impractical in production environments. They can capture organizationally relevant outcomes such as task throughput or code volume, but they are more exposed to confounding from team composition, rollout timing, and managerial process changes.[1]
- Meta-analyses and systematic reviews: These synthesize the field and are especially important because individual studies use different productivity markers. The May 2026 meta-analysis found a moderate average positive effect with substantial heterogeneity, while the ASE 2026 systematic review identified 187 distinct productivity and productivity-related markers, showing how dispersed the measurement landscape has become.[1] [14]
- Observational and longitudinal studies: These capture adoption at scale and over time, including shifts in work patterns, verification burden, and developer experience. They are weaker for causal attribution, but stronger for understanding how AI changes real workflows after the novelty period.[3] [4]

A central lesson from this landscape is that different methods often measure different layers of output. Task completion time, commits, lines of code, self-reported time savings, release frequency, maintainability, and skill retention are not interchangeable, which is why studies can all be internally valid yet still appear to disagree.[14] [4]

1.3

Scope, methodology, and how findings are evaluated in this report

This report is scoped narrowly to empirical evidence on productivity effects from AI coding assistants, with a deliberate separation between claimed benefits and independently measured outcomes. It is tool-agnostic, globally scoped, and focused on studies that quantify effects using observable outcomes such as task completion time, throughput, commits, releases, code quality proxies, maintainability indicators, or skill assessment results, while treating vendor surveys and marketing materials as contextual comparators rather than proof.[1] [4]

Findings in the report are evaluated using a hierarchy of evidentiary weight.

- Highest weight: Preregistered or well-specified randomized controlled trials, especially where outcomes are directly measured rather than self-reported.[10] [2]
- High weight: Meta-analyses and systematic reviews that aggregate multiple studies and explicitly assess heterogeneity or risk of bias.[1] [14]
- Moderate weight: Large-scale observational and longitudinal studies that reveal real-world usage patterns, perception gaps, and organizational frictions, but cannot fully isolate causality.[3] [4]
- Lower weight for causal claims: Vendor-commissioned surveys, self-reported savings, and internal case studies without transparent counterfactual design, which may still be useful for understanding adoption narratives and perceived value.[33]

The report also applies three interpretive rules when synthesizing results.

- Measured beats perceived: If self-assessments conflict with observed task time or output, the measured outcome is treated as primary evidence, as illustrated by the METR RCT and by longitudinal evidence showing stable perceived gains alongside worsening flow state and cognitive load.[2] [3]
- Downstream outcomes matter: Increases in coding activity are not assumed to translate directly into shipped software, maintainability, or learning, because the literature repeatedly distinguishes local task acceleration from system-level productivity.[1] [14]
- Context moderates effect size: Results are interpreted through developer seniority, task type, and environment, because the best available synthesis shows larger gains in controlled settings and smaller or mixed effects in enterprise and open source contexts.[1] [10] [2]

Taken together, this methodology is designed to answer a practical question for technology leaders: not whether AI coding assistants can help, but under what conditions measured gains are credible, how large they appear to be, and where current evidence remains too inconsistent to support strong operational claims. That framing is necessary because the field has already moved beyond simple adoption questions into a harder evaluation problem about throughput, quality, maintainability, and capability development over time.[1] [3] [33]

2

Vendor Claims and Marketing Figures

2

Vendor Claims and Marketing Figures

This section examines the most cited vendor productivity figures as marketing claims rather than as independent evidence. The central analytical issue is not whether these studies are fabricated, but whether their designs, outcome measures, and reporting choices justify broad operational conclusions for enterprise software development.[35] [50] [33]

Across the current vendor literature, the headline numbers are typically derived from narrow tasks, internal telemetry, observational usage data, or self-reported time savings rather than independently replicated field experiments. That does not make the claims useless, but it does mean they should be interpreted as upper-bound or context-bound indicators of potential value, not as settled estimates of average productivity impact across teams, seniority levels, and software delivery environments.[35] [OpenAI Productivity Note 1 [Jul 2025]] (https://cdn.openai.com/global-affairs/be0fe9e0-eb97-43d1-9614-99f2bd948bcc/OpenAI_Productivity-Note_Jul-2025.pdf) [58]

2.1

GitHub Copilot's "55% faster" claim and its methodological basis

GitHub Copilot's widely cited claim comes from a controlled experiment reported in both an academic paper and GitHub's own research communications. In that study, recruited developers were asked to build an HTTP server in JavaScript, and the treatment group with Copilot completed the task 55.8% faster than the control group.[35] [50]

Methodologically, the claim has real strengths.

- Experimental design: The result comes from a randomized controlled task study rather than a pure opinion survey, which gives it more causal credibility than most vendor marketing figures.[35]
- Direct outcome measure: The primary endpoint was task completion time, which is stronger than asking participants whether they felt faster.[35]
- Transparent headline result: The paper states the exact estimate of 55.8% faster, making the core claim traceable to a specific published experiment rather than an opaque internal dashboard.[35]

Its limitations are equally important when assessing external validity.

- Single-task scope: The experiment centered on one bounded programming task, which is materially narrower than real software work involving debugging, code review, architecture, coordination, and deployment.[35]
- Short-horizon measurement: The study measured immediate completion speed, not downstream outcomes such as defect rates, rework, maintainability, or release throughput.[35]
- Vendor proximity: The paper's authors included GitHub-affiliated researchers, which does not invalidate the result but does increase the need for independent replication before treating the estimate as representative.[35]

The most defensible interpretation is that the 55% figure demonstrates that large short-run gains are possible on well-scoped coding tasks under favorable conditions. It is less credible as a general estimate of end-to-end software engineering productivity, especially because later

independent studies reported smaller gains, wide uncertainty, or even slowdowns in more realistic settings.[35] [53]

2.2

OpenAI and Anthropic internal studies reporting up to 80% time savings

The strongest current "up to 80%" style claim in this area appears in Anthropic's January 29, 2026 research note on coding skills, which states that an observational study of Claude.ai data found AI can speed up some tasks by 80%. Anthropic itself presents this as a result for "some tasks," not as an average effect across software engineering work, and it appears in the context of a broader discussion that also highlights learning trade-offs from AI assistance.[33]

OpenAI's public materials available through current official sources are more restrained in wording. OpenAI's July 2025 productivity note reports 25% greater efficiency for consultants in a lab experiment using GPT-4 and cites self-reported time savings in other occupations, while its 2025 enterprise report says ChatGPT Enterprise users attribute 40 to 60 minutes saved per active day on average, with some functions reporting 60 to 80 minutes. [OpenAI Productivity Note 1 [Jul 2025]](https://cdn.openai.com/global-affairs/be0fe9e0-eb97-43d1-9614-99f2bd948bcc/OpenAI_Productivity-Note_Jul-2025.pdf) [58]

These claims have three recurring methodological constraints.

- Observational framing: Anthropic's 80% figure is described as coming from observational Claude.ai data, which is useful for identifying high-performing use cases but weaker than randomized designs for causal inference.[33]
- Selective task framing: "Up to 80%" is a maximum or best-case formulation, which can be accurate while still being unrepresentative of median outcomes.[33]
- Mixed outcome types: OpenAI's public evidence combines lab efficiency studies, enterprise self-reports, and usage telemetry, which are not interchangeable with measured software delivery outcomes. [OpenAI Productivity Note 1 [Jul 2025]](https://cdn.openai.com/global-affairs/be0fe9e0-eb97-43d1-9614-99f2bd948bcc/OpenAI_Productivity-Note_Jul-2025.pdf) [58]

Title — Vendor claims versus evidentiary basis

CLAIM SOURCE	HEADLINE FIGURE	EVIDENTIARY BASIS	MAIN LIMITATION
GitHub Copilot study	55.8% faster	Controlled task experiment	Single bounded coding task.
Anthropic coding note	Up to 80% faster on some tasks	Observational Claude.ai data	Best-case phrasing, not average effect.
OpenAI productivity note	25% more efficient	Lab experiment outside software engineering	Not a direct coding field study.
OpenAI enterprise report	40 to 60 minutes saved per active day on average	Self-reported enterprise survey data	Perceived savings, not independently measured output.

Source: [35] [33] [OpenAI Productivity Note 1 [Jul 2025]](https://cdn.openai.com/global-affairs/be0fe9e0-eb97-43d1-9614-99f2bd948bcc/OpenAI_Productivity-Note_Jul-2025.pdf) [58]

For decision-makers, the practical reading is that vendor studies identify plausible upside and commercially relevant use cases, but they do not establish a stable expected return for software teams. The more ambitious the claim, the more important it is to ask whether the number reflects a maximum observed case, a self-reported perception, or a directly measured causal effect in realistic engineering work.[33] [OpenAI Productivity Note 1 [Jul 2025]](https://cdn.openai.com/global-affairs/be0fe9e0-eb97-43d1-9614-99f2bd948bcc/OpenAI_Productivity-Note_Jul-2025.pdf) [58]

2.3

Why self-commissioned and self-reported figures warrant scrutiny

Self-commissioned and self-reported figures deserve scrutiny because they are structurally prone to optimistic bias even when reported in good faith. Vendors usually choose the task framing, participant pool, success metric, and headline statistic, which creates strong incentives to foreground the most favorable interpretation of mixed evidence.[35] [33]

Several specific risks recur across this literature.

- Selection effects: Internal studies may overrepresent motivated early adopters, simpler tasks, or workflows already well matched to the tool.[58] [53]
- Outcome substitution: Studies often report lines of code, task completion, or perceived time saved instead of shipped features, defect escape, maintainability, or long-run team capability.[35] [OpenAI Productivity Note 1 [Jul 2025]](https://cdn.openai.com/global-affairs/be0fe9e0-eb97-43d1-9614-99f2bd948bcc/OpenAI_Productivity-Note_Jul-2025.pdf).
- Framing bias: “Up to” claims highlight maxima, while average, median, variance, and failure cases may be less visible in marketing summaries.[33]
- Perception bias: Developers often report feeling faster or more productive even when measured outcomes are smaller or mixed, which is why self-report should not be treated as equivalent to observed performance.[50] [37]

Independent validation matters because it changes both the incentive structure and the evidentiary standard. External RCTs, preregistered studies, and large-scale observational analyses conducted outside vendor marketing functions are more likely to report null results, heterogeneous effects, and trade-offs that commercial narratives tend to compress.[37] [42]

The appropriate conclusion is not that vendor figures should be dismissed, but that they should be discounted until independently replicated under realistic conditions. For industry buyers, the right question is not whether a claim is impressive, but whether the underlying study design supports procurement, rollout, and workforce planning decisions at enterprise scale.[35] [33] [58]

3

What Controlled Experiments Actually Measured

3

What Controlled Experiments Actually Measured

Controlled experiments provide the clearest causal evidence in this literature because they compare AI-assisted and non-assisted conditions under defined tasks, participant pools, and outcome measures. As of August 25, 2026, the most decision-relevant RCT evidence does not support a single universal productivity number. Instead, it shows a spread from meaningful speedups in bounded enterprise or feature implementation tasks to measurable slowdowns in realistic open-source maintenance work, with uncertainty driven by context, developer experience, and what exactly is being measured.[10] [2] [72]

The common pattern across these experiments is that measured task time is often less favorable than perceived productivity. Google's enterprise RCT found a positive effect but with wide confidence intervals and explicit limits on generalization, while METR found the opposite of participant expectations, and the two-phase maintainability study found short-run speed gains without downstream maintainability gains. For industry buyers, the implication is that controlled evidence supports conditional usefulness, not broad vendor-style claims of uniform acceleration across software engineering work.[10] [2] [72]

3.1

Google RCT (96 engineers): ~21% task completion speedup with wide confidence intervals

Google's enterprise-based randomized controlled trial tested 96 full-time software engineers on a complex internal task using three AI features and measured time on task rather than self-reported satisfaction. The study's best estimate was that AI shortened time on task by about 21%, making it one of the stronger causal results in favor of productivity gains, but the authors also state that the confidence interval was large and that the result should not be assumed to transfer across tools, organizations, or time periods.[10]

- What was actually measured: Direct task completion time on a complex enterprise-grade assignment, not commits, lines of code, or perceived speed.[10]
- Why the result matters: It is a genuine randomized experiment in a professional workplace population rather than a student lab sample or vendor survey, which gives the estimate more causal credibility than most marketing claims.[10]
- Why caution is still required: The paper explicitly warns that the effect size came from internal Google tooling in summer 2024 and may not generalize broadly, especially because the interval around the estimate is wide.[10]

Methodologically, this is rigorous evidence of possible short-run acceleration under enterprise conditions, but not proof of a stable average uplift for software engineering as a whole. It is best interpreted as evidence that AI can reduce completion time on certain complex internal tasks, while leaving substantial uncertainty about the true magnitude and portability of that gain.[10]

3.2

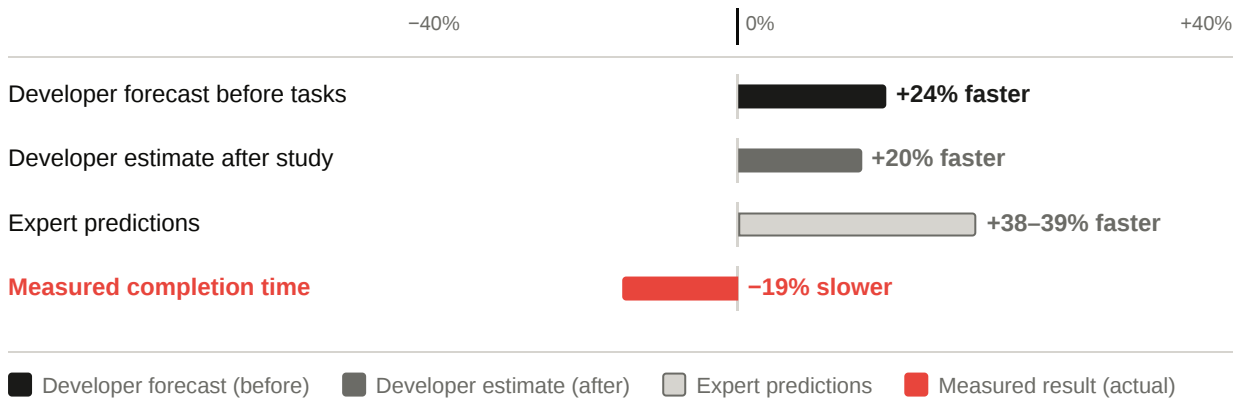
METR RCT (experienced open-source developers): 19% slowdown despite perceived gains

The METR randomized controlled trial is the strongest current counterexample to optimistic productivity narratives because it studied experienced open-source developers working on mature projects they already knew well. Across 246 tasks completed by 16 developers with an average of about five years of prior experience on their projects, allowing frontier 2025 AI tools increased completion time by 19% rather than reducing it.[2]

Title — Perceived versus measured outcomes in the METR RCT

MEASURE	RESULT	INTERPRETATION
Developer forecast before tasks	24% faster	Expected substantial gain.
Developer estimate after study	20% faster	Perceived benefit remained positive.
Measured completion time	19% slower	Actual productivity declined.
Expert predictions	38% to 39% shorter	External observers were also too optimistic.

Source: [2]



- What was actually measured: Completion time on real tasks in mature open-source repositories, with each task randomly assigned to allow or disallow AI use.[2]
- Why this result is especially important: It isolates a setting that is closer to high-context maintenance and extension work than to greenfield coding demos, which makes it highly relevant for senior developers and established codebases.[2]
- What it reveals about perception: Participants believed AI helped both before and after the experiment, even though measured performance worsened, making this one of the clearest demonstrations that subjective productivity can diverge sharply from objective task time.[2]

The authors examined multiple possible artifacts and concluded that, while no experiment can rule out every design limitation, the slowdown appeared robust across their analyses. For decision-makers, the key lesson is that experienced developers in high-context environments may incur verification, navigation, and integration overheads that outweigh generation speed, especially when the work is not primarily boilerplate.[2]

3.3

Two-phase preregistered RCT (151 developers): 30.7% median speedup in Phase 1, no maintainability impact in Phase 2

The two-phase preregistered maintainability study adds an important layer missing from most productivity experiments by separating immediate coding speed from downstream code evolution. In the reported results paper, 151 participants, 95% of them professional developers, first implemented a feature in a Java web application with or without AI assistance, and then a different set of participants evolved those resulting codebases without AI in a randomized Phase 2 design.[69] [72]

- Phase 1 finding: AI assistance produced a 30.7% median reduction in completion time, which is a substantial short-run speedup and broadly consistent with the upper end of favorable controlled task studies.[72]
- Phase 2 finding: When new developers later evolved the code without AI, the study found no significant differences in completion time or code quality, and Bayesian analysis suggested any downstream effects were at most small and highly uncertain.[72]
- Why this design matters: It tests not only whether AI helps the original author move faster, but whether that speed leaves a measurable maintainability signature for the next developer, which most vendor studies do not examine.[69] [72]

This study therefore supports a narrower conclusion than either enthusiasts or critics often claim. It provides credible evidence that AI can accelerate initial implementation in a controlled feature development task, but it does not show that this acceleration either improves or degrades downstream maintainability in a measurable way under the study's tasks and metrics.[72]

4

Quasi-Experiments and Large-Scale Observational Data

4

Quasi-Experiments and Large-Scale Observational Data

Quasi-experiments and large-scale observational studies add an important layer that controlled task experiments cannot capture: they observe AI coding assistance inside production workflows, where coordination, review, release management, and organizational hierarchy shape whether local coding gains become business-relevant output. In this evidence tier, the recurring pattern is that AI often increases visible engineering activity more than it increases shipped software, suggesting that code generation is only one stage in the delivery pipeline rather than a direct proxy for end-to-end productivity.[90] [102] [3]

Compared with the RCTs discussed earlier, these studies are weaker on causal isolation but stronger on ecological validity. They are especially useful for technology decision-makers because they show where gains concentrate, who benefits most, and why throughput metrics such as lines of code or commits can overstate realized delivery impact once human review, integration, and release gates are taken into account.[91] [90] [78]

4.1

Ant Group / CodeFuse quasi-experiment: 50% more lines of code, 22% more tasks (junior devs)

The Ant Group CodeFuse study is one of the clearest field-style quasi-experiments in this literature because it compares a treatment group given access to CodeFuse with a matched control group that was not informed about the tool during a real organizational rollout. The study reports that generative AI increased code output by more than 50% on average, but it also explicitly tests a more robust task-based measure and finds a 22% increase in tasks completed, which is more decision-relevant than raw code volume alone.[102] [91]

- Measured effect: Code output rose by over 50% on average in the treatment group relative to the matched control group.[102]
- Robustness check: Because LLMs can inflate code volume, the authors also evaluated completed tasks and found a 22% increase, reducing the risk that the result is merely verbosity masquerading as productivity.[91] [102]
- Distribution of gains: The statistically significant gains were concentrated among junior or entry-level developers, while effects for more senior developers were limited and not statistically significant in the reported field experiment.[91] [102]

For this report's measured-versus-claimed framing, the Ant result is important because it shows that favorable enterprise outcomes do exist outside vendor marketing studies, but they are conditional. The strongest interpretation is not that AI uniformly makes all developers 22% to 50% more productive, but that it can materially raise throughput in coding-heavy work, especially for less experienced developers whose tasks are more amenable to generation and completion support.[102] [91]

4.2

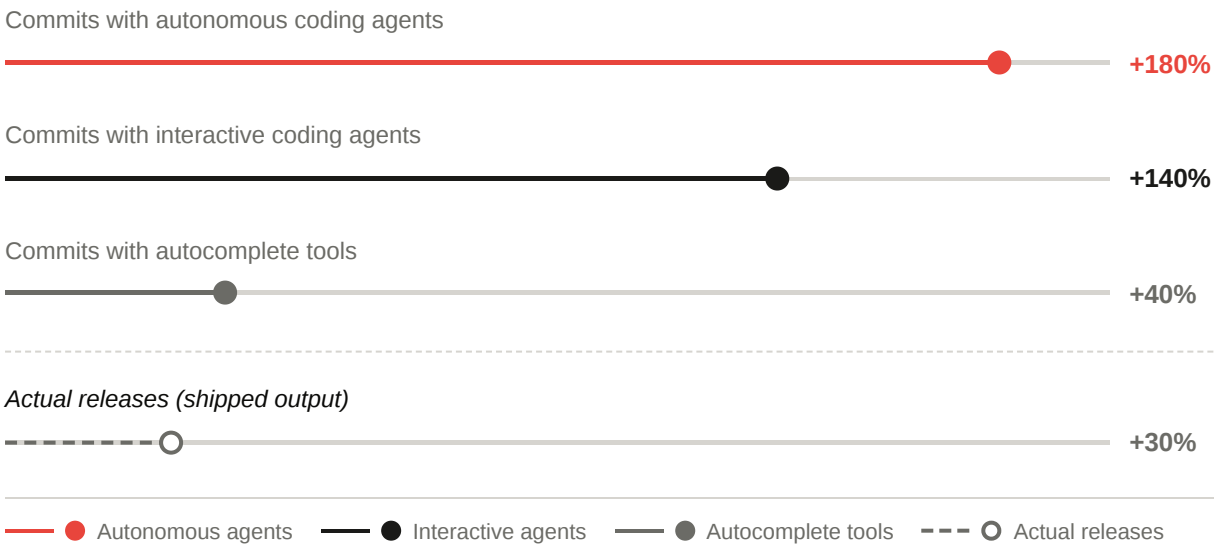
NBER study (100,000+ GitHub developers): commit activity up 40-180% by tool generation, but releases up only 30%

The NBER working paper extends the evidence base from team-level field experiments to platform-scale observational data by combining AI usage telemetry with activity from more than 100,000 GitHub developers. Its central finding is that newer generations of AI coding tools produce progressively larger increases in coding activity, but those gains compress sharply as work moves from commits to projects and then to actual releases.[90]

Title — Activity gains versus release gains in the NBER study

METRIC	REPORTED GAIN	INTERPRETATION
Commits with autocomplete tools	40%	Modest activity uplift at the lowest automation tier.
Commits with interactive coding agents	140%	Much larger increase in coding activity.
Commits with autonomous coding agents	180%	Very large increase in code production activity.
Actual releases	30%	Downstream shipped output rises, but far less than commit activity.

Source: [90]



- Production hierarchy effect: The paper reports cumulative commit effects of 40%, 140%, and 180% for autocomplete, interactive agents, and autonomous agents respectively, but notes that these gains attenuate to 50% for number of projects and to 30% for actual releases.[90]
- Why this matters: It is one of the strongest current demonstrations that coding activity metrics and shipped software metrics are not interchangeable. A team can generate far more commits

without achieving a proportional increase in production output.[90]

- Analytical implication: The study does not show that AI has little value. It shows that value decays across the delivery funnel, so the business effect depends on how efficiently organizations convert generated code into reviewed, integrated, and releasable software.[90]

This makes the NBER result especially important for executives who are tempted to use commit counts, pull request volume, or lines of code as rollout KPIs. Those indicators may capture local acceleration, but the paper suggests they can materially overstate realized delivery gains if the rest of the software production system does not scale with the new rate of code generation.[90] [3]

4.3

The 'human bottleneck' problem: why activity gains do not translate linearly into shipped software

Taken together, the Ant and NBER findings point to a structural constraint that this report treats as the human bottleneck problem. AI can accelerate code drafting and increase visible activity, but software delivery still depends on human-intensive stages such as prompt steering, verification, review, integration, prioritization, security checking, and release approval, which do not scale automatically with generation speed.[90] [3] [78]

- Verification overhead: The 2026 longitudinal study finds a shift from creation toward verification activities and introduces the idea of supervisory engineering work, meaning developers spend more effort directing, evaluating, and correcting AI output rather than simply writing code.[3]
- Interaction constraint: The July 2026 human-centered agents paper argues that practical usefulness is increasingly limited by task alignment, verifiability, steerability, and adaptability rather than raw task-solving capability alone.[78]
- Funnel compression: The NBER paper provides the clearest quantitative expression of this bottleneck, showing that very large gains at the commit layer shrink substantially by the time work reaches projects and releases.[90]

The implication for software release efficiency is straightforward. AI reduces the cost of producing candidate code, but it does not remove the need for human judgment about correctness, maintainability, security, architectural fit, and release readiness, so the constraint moves downstream rather than disappearing.[90] [78]

For decision-makers, this means observed activity gains should be interpreted as upstream throughput gains, not as direct evidence of proportional business output. Unless organizations also improve review capacity, test automation, integration discipline, and release governance, the likely result is more code entering the pipeline faster than humans can confidently validate and ship it.[90] [3] [78]

5

Meta-Analysis and Systematic Review Findings

5

Meta-Analysis and Systematic Review Findings

At the synthesis level, the evidence base becomes clearer on one point and more cautious on another. The clearest point is that AI coding assistants show a positive average productivity effect across studies; the caution is that this average conceals large variation in task design, participant experience, outcome definitions, and work context, which limits how confidently any single headline number can be operationalized across enterprises.[1] [14]

The two most decision-relevant synthesis studies available by August 25, 2026, point in the same strategic direction. The May 2026 meta-analysis reports a statistically significant but moderate pooled productivity effect with substantial heterogeneity, while the ASE 2026 systematic review shows that the field is measuring productivity through a highly fragmented set of 187 markers, with objective outcomes remaining mixed rather than uniformly positive.[1] [14]

For industry readers, the practical implication is that synthesis evidence narrows the plausible range of claims without eliminating uncertainty. It supports the view that AI assistance often helps under bounded conditions, but it does not support treating vendor-style speedup figures as stable expectations for mature enterprise delivery systems.[1] [4]

5.1

May 2026 meta-analysis of 23 studies: moderate positive effect (Hedges' g = 0.33)

The May 2026 meta-analysis is currently the strongest quantitative synthesis of the field because it aggregates 23 studies and 27 effect sizes spanning publications from 2019 through 2025, using standardized effect sizes and formal risk-of-bias procedures. Its headline result is a statistically significant moderate positive productivity effect of Hedges' g = 0.33 with a 95% confidence interval of [0.09, 0.58], which supports a real average benefit while also indicating that the central effect is materially smaller than the most aggressive vendor claims.[1]

- Scope: The study systematically searched ACM, arXiv, Scopus, and Web of Science for quantitative comparisons of AI-assisted versus unassisted programming.[1]
- Outcomes included: Productivity was operationalized through measures such as task completion time, commits, and lines of code, which already signals that the pooled estimate combines non-equivalent proxies.[1]
- Bias handling: The authors report using RoB2 and ROBINS-I to assess risk of bias, which strengthens the credibility of the synthesis relative to narrative reviews.[1]

The most important qualifier is heterogeneity. The authors explicitly report substantial heterogeneity and note that productivity gains tend to be larger in controlled experimental settings, while effects are smaller in open-source and enterprise contexts, meaning the pooled average should be read as a cross-context summary rather than a forecast for any specific organization.[1]

This matters because a moderate pooled effect can coexist with both strong positive and negative individual studies. The meta-analysis is therefore best interpreted as evidence against the claim that AI coding assistants are mostly ineffective, but equally as evidence against the claim that large speedups are typical across realistic software engineering environments.[1] [2]

The same paper also reports no statistically significant effect on learning outcomes, with Hedges' g = 0.14 and a 95% confidence interval of [-0.18, 0.47]. That finding is outside the narrow productivity question, but it reinforces the broader conclusion that short-run coding acceleration should not be assumed to translate into broader capability gains.[1]

5.2

ASE 2026 systematic review: 187 productivity markers identified, mixed objective outcomes

The ASE 2026 systematic review adds a different kind of value from the meta-analysis: it explains why the literature is so hard to compare in the first place. According to the conference paper abstract, the review identifies 187 unique productivity and productivity-related markers, which indicates that the field lacks a stable shared measurement framework and is instead evaluating AI coding assistants through a wide assortment of local proxies.[14]

Title — Productivity marker fragmentation in the ASE 2026 review

FINDING	REPORTED VALUE	WHY IT MATTERS
Unique productivity markers identified	187	Shows extreme measurement fragmentation.
Overall evidence pattern	Mixed results	No single consistent productivity story.
Objective outcomes	Mixed	Measured effects vary by metric and context.
Evidence base characterization	Highly fragmented	Limits direct comparison across studies.

Source: [14]

The review's core contribution is conceptual as much as empirical. If one study measures completion time, another measures commits, another measures lines of code, and another measures self-reported satisfaction or perceived speed, then "productivity" becomes an umbrella label rather than a single comparable outcome.[14] [4]

- Measurement problem: A fragmented marker landscape makes it easy for favorable studies to emphasize upstream activity metrics while less favorable downstream metrics remain underreported.[14]
- Interpretation problem: Mixed objective outcomes mean that positive self-reports or code-volume gains cannot be assumed to imply faster delivery, better maintainability, or stronger learning.[14] [3]
- Decision-making problem: Organizations that benchmark vendors against only one local KPI risk overstating value because the literature does not support any single proxy as a sufficient measure of engineering productivity.[4] [14]

In effect, the ASE review explains why apparently contradictory studies can all be methodologically defensible. They are often measuring different slices of work, at different time horizons, with different populations, so mixed objective outcomes are not noise around a single truth but evidence that productivity effects are multidimensional and context-sensitive.[14] [105]

5.3

Why controlled settings consistently outperform real-world enterprise results

The synthesis evidence suggests that controlled settings outperform real-world enterprise results for structural rather than accidental reasons. Controlled experiments usually isolate a bounded coding task, minimize coordination overhead, shorten the evaluation horizon, and measure immediate completion speed, all of which favor tools that are strongest at local generation and suggestion.[1] [10]

By contrast, enterprise software delivery includes review queues, integration constraints, security checks, architecture conformance, testing obligations, and release governance. These downstream stages absorb part of the local coding gain and can even reverse it when verification and correction costs exceed drafting savings, which is consistent with both the meta-analysis finding of smaller enterprise effects and the broader literature on supervisory engineering work.[1] [3]

- Task boundedness: Lab and RCT tasks are often cleaner, shorter, and more generation-friendly than production maintenance, debugging, or cross-team change work.[10] [2]
- Context load: Real repositories impose domain knowledge, architectural history, and integration constraints that reduce the value of generic code generation and increase verification effort.[2] [3]
- Metric compression: Controlled studies usually stop at task completion, while enterprise outcomes depend on whether code survives review, merges cleanly, avoids rework, and contributes to release throughput.[4] [1]

This pattern also helps reconcile why some experiments show meaningful speedups while field evidence remains mixed. AI is often effective at reducing the cost of producing candidate code, but enterprise productivity depends on the full socio-technical system, not just on the speed of first-draft generation.[1] [4]

For decision-makers, the implication is straightforward. Controlled-study gains should be treated as evidence of technical potential under favorable conditions, whereas enterprise rollout expectations should be calibrated to smaller and more variable realized effects unless the organization also improves review capacity, testing automation, and governance around AI-generated code.[1] [3]

6

The Perception-Reality Gap

6

The Perception-Reality Gap

This section examines one of the most consistent findings in the 2025 to 2026 evidence base: developers often report strong productivity gains from AI coding assistants even when measured outcomes are smaller, mixed, or negative. The gap matters because procurement and rollout decisions are frequently influenced by perceived speed, satisfaction, and visible code output, while the more decision-relevant outcomes include completion time, verification burden, cognitive load, and the ability to sustain focused work over time.[3] [2] [105]

Across the strongest studies discussed in this report, the perception-reality gap is not a minor measurement artifact but a recurring empirical pattern. Longitudinal evidence shows stable self-reported productivity gains alongside worsening flow state and cognitive load, while the METR randomized trial shows that experienced developers expected a substantial speedup and still felt helped afterward despite being measurably slower, indicating that subjective experience and objective throughput can diverge in systematic ways.[3] [2]

6.1

Longitudinal study (158 developers, 6 months): 82-84% perceived gains, but declining flow state and worsening cognitive load

The May 2026 longitudinal mixed-methods study provides one of the clearest demonstrations that perceived productivity can remain positive even as the quality of the work experience deteriorates. The study surveyed 158 eligible participants at the first time point, 101 at the second, and a matched cohort of 95 across a six-month interval, making it more informative about persistence effects than one-shot surveys or short lab tasks.[3]

- Perceived gains remained high: 82% reported spending less time on writing code, and 84% reported productivity improvement at both time points, indicating that the positive productivity narrative remained stable over time rather than fading after initial novelty.[3]
- Experience quality worsened: In the matched cohort, the share reporting worsened developer experience in at least one dimension nearly doubled from 14% to 27%, with the paper specifically identifying erosion in flow state and cognitive load even as feedback loops improved.[3]
- Work shifted rather than simply accelerated: The authors report a broader movement from creation toward verification activities and introduce the concept of supervisory engineering work, meaning developers increasingly direct, evaluate, and correct AI output instead of only producing code directly.[3]

This is analytically important because it suggests that developers may interpret reduced typing effort or faster first drafts as overall productivity improvement, even when the saved effort is partly replaced by monitoring, checking, and integration work. In other words, the tool can feel helpful at the interaction level while simultaneously making sustained concentration harder and increasing the mental overhead required to trust the result.[3] [105]

The study's productivity experience paradox therefore sharpens a key distinction used throughout this report: feeling faster is not the same as being faster, and being faster is not the same as working under lower cognitive strain. For industry decision-makers, this means self-reported satisfaction or time saved should not be treated as sufficient evidence of durable

productivity improvement unless accompanied by direct measures of task time, review burden, and developer experience over time.[3]

6.2

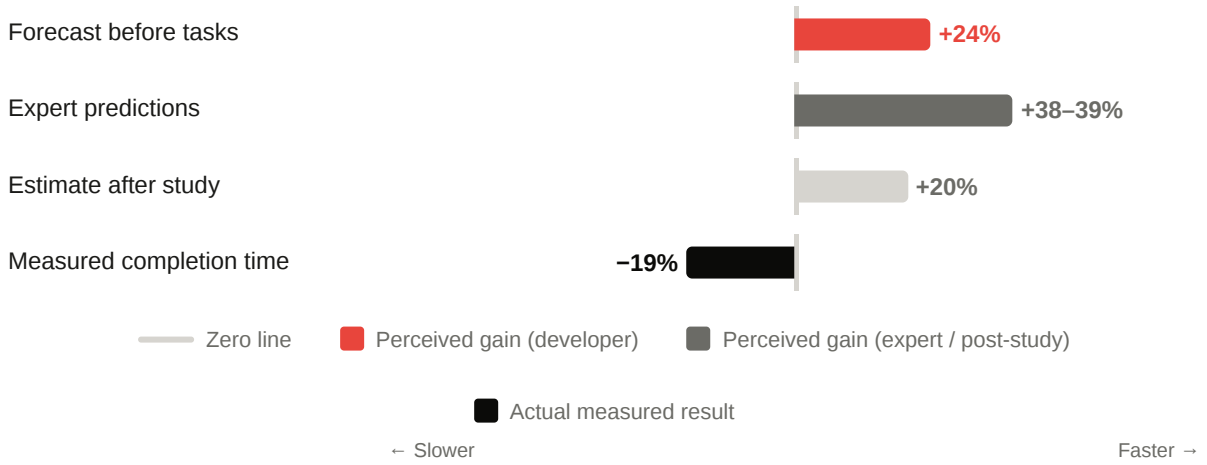
METR finding: developers estimated 24% speedup, measured 19% slowdown

The METR randomized controlled trial is the strongest direct test of self-assessment accuracy in this literature because it compares developer expectations with measured task completion time under randomized AI-allowed versus AI-disallowed conditions. In mature open-source projects, 16 experienced developers completed 246 tasks on repositories they knew well, and the measured result was a 19% increase in completion time when AI tools were allowed.[2]

Title — Perception versus measurement in the METR trial

MEASURE	RESULT	INTERPRETATION
Forecast before tasks	24% faster	Developers expected a substantial gain.
Estimate after study	20% faster	Positive perception persisted after use.
Measured completion time	19% slower	Actual productivity declined.
Expert predictions	38% to 39% shorter	External observers were also too optimistic.

Source: [2]



- The misalignment was not limited to ex ante optimism: participants forecast a 24% speedup before starting tasks and still estimated a 20% speedup after completing the study, despite the measured slowdown.[2]
- The bias was socially shared, not merely individual: experts in economics predicted 39% shorter completion times and machine learning experts predicted 38% shorter times, showing that overestimation extended beyond the developers themselves.[2]
- The setting was especially relevant to senior engineering work: tasks were drawn from mature projects where participants had an average of about five years of prior experience, so the result is highly salient for high-context maintenance and extension work rather than greenfield demos.[2]

The most plausible interpretation is not that developers are irrational, but that they are using the wrong internal proxy for productivity. Faster idea generation, reduced blank-page friction, and the salience of occasional high-quality suggestions can create a strong subjective sense of acceleration, while the hidden costs of prompt iteration, output validation, repository navigation, and correction accumulate more quietly in elapsed time.[2] [3]

For this report's measured versus claimed framework, the METR result is pivotal because it shows that even experienced developers working in familiar codebases can misread their own net productivity effect. That makes self-reported speedup a weak standalone basis for enterprise ROI claims, especially in complex environments where verification and integration dominate the total cost of task completion.[2]

6.3

Why developers consistently overestimate productivity gains from AI tools

The current evidence suggests that overestimation is structural rather than accidental. Developers are often highly accurate about one local fact, namely that AI reduces the effort of producing candidate code, but they are less accurate about the full workflow cost once checking, steering, reconciling, and integrating that code are included.[3] [2]

Several mechanisms recur across the literature.

- Salience bias: Developers vividly notice moments when the tool produces a useful block instantly, but they less readily attribute time lost to reviewing weak suggestions, reformulating prompts, or debugging subtly wrong output. This can inflate retrospective judgments of speed.[2]
- Effort substitution: AI often removes keystrokes and search effort while adding supervisory work. Because the new work feels lighter or more conversational than manual coding, users may perceive net improvement even when total elapsed time does not improve.[3]
- Cognitive offloading illusion: Offloading first-draft generation can create a sense of fluency and momentum, but fluency is not equivalent to throughput. The 2025 systematic review identifies concerns around cognitive offloading and inconsistent effects on code quality, reinforcing that easier interaction can coexist with mixed objective outcomes.[105]
- Metric confusion: Developers often judge productivity through local indicators such as typing less, finding examples faster, or finishing substeps sooner, whereas organizations care about end-to-end completion time, review load, maintainability, and release throughput. The mismatch between local and system-level metrics encourages optimistic self-assessment.[3] [2]

A further reason is that AI changes the phenomenology of work. When developers move from writing every line themselves to curating and validating generated output, the work can feel faster because the visible act of production is compressed, even though the invisible act of judgment expands. That is consistent with the longitudinal study's finding that feedback loops improved while flow state and cognitive load worsened, suggesting a trade in which responsiveness increases but mental fragmentation also rises.[3]

The practical implication is that organizations should treat developer enthusiasm as a useful adoption signal but not as a performance metric. If teams want to know whether AI tools are genuinely improving productivity, they need direct measurement of cycle time, rework, review burden, and task completion in representative workflows, because subjective impressions alone systematically overstate realized gains in the current evidence base.[2] [3] [105]

7

Skill Degradation and Learning Trade-offs

7

Skill Degradation and Learning Trade-offs

The productivity literature reviewed so far shows that short-run acceleration and long-run capability are not the same outcome. The evidence most relevant to skill retention is narrower than the productivity evidence, but what exists is directionally consistent: AI assistance can reduce immediate effort while weakening mastery of the concepts and reasoning steps that developers would otherwise practice directly.[135] [1]

For decision-makers, this section matters because software capability compounds through repeated problem solving, code review, debugging, and explanation inside teams. If AI shifts work from active construction toward supervision and verification, organizations may gain local speed while gradually thinning the depth of individual understanding and the amount of tacit knowledge that propagates through the team.[3] [24]

7.1

Anthropic's RCT: minimal speed gain, but AI users scored 50% on skill quiz vs. 67% for non-AI users

Anthropic's January 29, 2026 study is one of the few controlled experiments to measure both immediate task performance and near-term skill formation in coding. The study split participants into a warm-up, a main coding task using Trio-related asynchronous programming concepts, and a follow-up quiz on concepts they had just used, allowing the authors to compare output speed with retained understanding.[135]

- Core result: AI assistance produced only a minimal productivity gain of roughly two minutes on the task, but the AI-assisted group scored 50% on the skill quiz versus 67% for the non-AI group, a 17 percentage point gap that Anthropic describes as a 17% decrease in mastery and roughly the equivalent of nearly two letter grades.[24] [135]
- Statistical strength: Anthropic reports the quiz difference as statistically significant, with Cohen's d = 0.738 and p = 0.01, which is a non-trivial effect size for a short-horizon intervention and materially stronger than the observed speed benefit.[24]
- Interpretation: The study does not show that AI always harms learning, but it does show that when assistance substitutes for active reasoning during implementation, developers can complete the task with weaker conceptual retention immediately afterward.[135]

Title — Anthropic RCT productivity and mastery contrast

MEASURE	AI GROUP	NON AI GROUP	INTERPRETATION
Task speed gain	About 2 minutes faster	Baseline	Minimal immediate productivity benefit.
Skill quiz score	50%	67%	Lower retained understanding with AI use.
Mastery difference	-17 percentage points	Baseline	Meaningful short-run skill loss.

MEASURE	AI GROUP	NON AI GROUP	INTERPRETATION
Reported significance	d = 0.738, p = 0.01	Not applicable	Quiz gap is statistically significant.

Source: [24]

A second implication is methodological. Because the quiz covered concepts participants had just used minutes earlier, the result is not about distant forgetting but about weaker encoding during the task itself, which is precisely the mechanism organizations should worry about if AI becomes the default path for routine implementation work.[1][3][5]

7.2

Longitudinal cognitive load findings and the "productivity-experience paradox"

The six-month longitudinal mixed-methods study adds an important complement to Anthropic's RCT because it tracks what happens after the novelty period. Across 158 eligible participants at the first wave, 101 at the second, and a matched cohort of 95, perceived productivity remained high, but developer experience worsened on dimensions tied to sustained cognition rather than raw output.[3]

- Stable perceived gains: 82% reported spending less time writing code and 84% reported productivity improvement at both time points, showing that the positive productivity narrative persisted over six months rather than collapsing after initial enthusiasm.[3]
- Worsening experience: In the matched cohort, the share reporting worsened developer experience in at least one dimension rose from 14% to 27%, with the paper specifically identifying declining flow state and worsening cognitive load despite improved feedback loops.[3]
- Work substitution: The authors describe a shift from creation toward verification and supervisory engineering work, meaning developers increasingly spend effort directing, checking, and correcting AI output rather than constructing solutions end to end themselves.[3]

This is the productivity-experience paradox in its clearest form: the tool can feel productive because it reduces blank-page friction and accelerates first drafts, while simultaneously making work more fragmented and mentally effortful. That pattern aligns with the broader 2026 meta-analysis, which found no statistically significant overall learning benefit from AI assistance, with Hedges' g = 0.14 and 95% CI [-0.18, 0.47], suggesting that short-run convenience does not reliably translate into stronger skill development.[1]

The broader inference, supported but not directly proven by these studies, is that cognitive load may be moving rather than disappearing. Less effort is spent generating syntax and scaffolding, but more effort is spent evaluating uncertain outputs, maintaining context, and deciding when to trust the tool, which can degrade flow even when local task steps feel easier.[3][1][2][4]

7.3

Implications for long-term developer capability and team knowledge depth

For organizations, the central risk is not that developers will suddenly forget how to code. It is that repeated reliance on AI for routine implementation may reduce the frequency with which

engineers rehearse the reasoning patterns that build durable expertise, especially around debugging, API semantics, concurrency, edge cases, and architectural trade-offs.[24] [1]

- Individual capability risk: Anthropic's RCT suggests that when AI substitutes for active problem solving, developers may finish with weaker mastery of concepts they just applied, which raises concern about cumulative underlearning if this pattern repeats across months of daily work.[135]
- Team knowledge risk: If more implementation is delegated to AI and less reasoning is externalized among humans, teams may generate less teachable context through design discussion, code walkthroughs, and peer explanation, reducing the spread of tacit knowledge across the organization. This is an inference from the observed shift toward supervisory work and should be treated as a plausible organizational mechanism rather than a directly measured outcome.[3] [130]
- Seniority amplification: The risk is likely highest for junior developers, because they benefit most from speed assistance in several productivity studies yet also have the most skill still to build. Anthropic explicitly warns that productivity benefits may come at the cost of the skills needed to validate AI-written code if early-career engineers' development is stunted.[24] [102]

The strategic implication is that AI coding tools should be governed not only as productivity systems but also as learning systems. Teams that want durable capability should preserve deliberate opportunities for unaided implementation, explanation-first prompting, code review that asks for reasoning rather than just acceptance, and rotation through debugging and architecture tasks where understanding must be demonstrated rather than inferred from shipped output.[24] [34]

In practical terms, the evidence supports a narrower conclusion than either strong optimism or strong pessimism. AI assistance can improve local throughput in some settings, but the current measured evidence does not justify assuming that those gains are capability-neutral over time, and it gives concrete reasons to monitor skill retention, review quality, and knowledge diffusion alongside speed metrics.[1] [135] [3]

8

Variation by Developer Experience and Task Type

8

Variation by Developer Experience and Task Type

The empirical record reviewed so far becomes more coherent when results are segmented by who is using the tool, what kind of work they are doing, and in which delivery environment the work occurs. Across the strongest studies available by August 25, 2026, developer experience appears to be the most powerful moderator: junior developers more often show measurable throughput gains, while senior developers in high-context codebases more often absorb verification, review, and rework costs that compress or reverse those gains.[105] [2] [73]

The same pattern holds across task categories and work settings. AI coding assistants are strongest where the task is locally specifiable and easy to verify, such as boilerplate generation, code completion, and bounded feature implementation, and weakest where success depends on deep repository knowledge, architectural judgment, debugging, or review. Laboratory and bounded enterprise experiments therefore tend to report larger gains than open-source maintenance settings, where context load and quality standards are higher and where expert developers must validate generated output against a long project history.[10] [69] [3]

8.1

Junior vs. senior developer outcomes: why experience level is the strongest moderating factor

Experience level is the clearest dividing line in the current evidence because AI changes the composition of work differently for novices and experts. Less experienced developers often spend more time on syntax recall, API discovery, scaffolding, and routine implementation, so they can benefit directly from generation and completion support; more experienced developers spend a larger share of time on integration, debugging, review, architecture, and exception handling, where AI output must be checked against richer mental models and stricter standards.[105] [4]

The strongest field evidence in favor of junior gains comes from the Ant Group and CodeFuse rollout, which found more than 50% higher code output and 22% more tasks completed, with statistically significant gains concentrated among junior or entry-level developers rather than senior staff. That result is important because it suggests AI can narrow execution bottlenecks for less experienced engineers, but it does not imply equal benefit across the team hierarchy.[102] [155]

The strongest countervailing evidence comes from experienced-developer settings. In the METR randomized controlled trial, 16 experienced open-source developers working on 246 real tasks in mature repositories they knew well were 19% slower with AI allowed, despite forecasting a 24% speedup beforehand and estimating a 20% speedup afterward.[2]

A related open-source observational study sharpens the mechanism behind this divergence. After GitHub Copilot adoption in OSS projects, productivity gains were primarily driven by less-experienced peripheral developers, while more experienced core developers reviewed 6.5% more code and experienced a 19% drop in their own original code productivity, indicating that AI can redistribute work upward toward expert maintenance and review.[73]

Title — Experience level as a productivity moderator

EVIDENCE SOURCE	JUNIOR OR LESS EXPERIENCED DEVELOPERS	SENIOR OR EXPERIENCED DEVELOPERS	MAIN IMPLICATION
Ant Group and CodeFuse field experiment	Largest measured gains, including 22% more tasks completed	Limited or non-significant gains reported for seniors	AI helps most where work is execution-heavy.
METR RCT in mature OSS projects	Not the study focus	19% slowdown on 246 tasks	High-context expert work can negate drafting gains.
OSS maintenance burden study	Peripheral developers drive output gains	Core developers review 6.5% more and lose 19% original-code productivity	Gains can be transferred into expert rework.
BNY Mellon mixed-method study	More likely to value immediate assistance and speed	More likely to emphasize ownership, expertise, and long-term metrics	Seniority changes what counts as productivity.

Source: [102] [2] [73] [4]

Why experience dominates other moderators is therefore straightforward. Senior developers are not merely faster coders; they are also the system's validators, integrators, and architectural gatekeepers, so any increase in low-cost code generation can create additional downstream work for them.[73] [3]

This also explains why self-reported gains can remain positive across experience levels even when measured outcomes diverge. Junior developers may correctly perceive reduced friction on routine tasks, while senior developers may still feel locally helped by drafting support even as total elapsed time rises because they must verify, adapt, and defend the output against production constraints.[2] [3]

8.2

Task-type breakdown: where AI helps (boilerplate, autocomplete) vs. where it does not (debugging, architecture, review)

The task-level evidence is more consistent than the headline productivity debate suggests. AI coding assistants perform best on tasks with three properties: the output format is familiar, the local objective is clear, and correctness can be checked quickly. That is why the literature repeatedly reports benefits for autocomplete, boilerplate generation, routine implementation, documentation support, and code search reduction, while results are weaker or mixed for debugging, design, architecture, and review.[105] [3]

The favorable controlled studies fit this pattern closely. GitHub Copilot's original 55.8% faster result came from a bounded HTTP server implementation task, Google's RCT found about a 21% reduction in time on a complex but still well-scoped enterprise task, and the two-phase preregistered RCT found a 30.7% median speedup in initial feature implementation. These are all tasks where producing a first working version is a large share of the total effort.[35] [10] [69]

By contrast, the less favorable studies are concentrated in tasks where generation is only the beginning of the work. The 2025 systematic review explicitly lists debugging and design among the task areas studied, but reports inconsistent effects on code quality and concerns

around cognitive offloading. The six-month longitudinal study similarly finds that developers report spending less time on writing code while shifting effort toward verification, evaluation, and correction, which implies that AI often substitutes for drafting rather than for diagnosis or judgment.[105] [3]

The METR RCT is especially informative here because its tasks came from mature open-source projects rather than synthetic coding exercises. In that setting, developers were slower with AI, which strongly suggests that debugging, repository navigation, edge-case handling, and integration into an existing architecture can outweigh the speed of code generation itself.[2]

A practical task taxonomy emerges from the evidence.

- Boilerplate and scaffolding: Usually favorable because the work is repetitive, pattern-based, and easy to inspect.[35] [105]
- Autocomplete and local code completion: Often favorable because the tool reduces keystrokes and search effort without requiring large architectural commitments.[35] [10]
- Bounded feature implementation: Frequently positive in experiments because the task is specifiable and success is measured at completion time.[10] [69]
- Debugging and fault isolation: Mixed to weak because the hard part is often causal diagnosis, not code emission.[105] [2]
- Architecture and design: Weakly supported because these tasks require trade-off reasoning, system context, and long-horizon judgment that current productivity studies rarely show AI improving directly.[105] [4]
- Code review and maintainability work: Often unfavorable for senior staff because generated code increases inspection and rework burden even when initial drafting is faster.[73] [3]

The key analytical point is that AI helps most when software work resembles text completion with local validation, and least when it resembles diagnosis, negotiation, or systems reasoning. This is why organizations that benchmark only on coding speed can overestimate value: they are measuring the part of the workflow where the tool is strongest and ignoring the parts where human judgment remains the bottleneck.[4] [3]

8.3

Enterprise vs. open-source vs. laboratory contexts and their divergent results

Context explains much of the apparent contradiction across studies because enterprise, open-source, and laboratory settings expose different mixes of task boundedness, repository familiarity, quality standards, and coordination overhead. The May 2026 meta-analysis explicitly reports substantial heterogeneity and notes that productivity gains tend to be larger in controlled experimental settings and smaller in open-source and enterprise contexts, which is the broadest synthesis-level confirmation of this pattern.[1]

Laboratory and tightly bounded experimental contexts usually produce the most favorable results because they isolate first-pass implementation and minimize downstream friction. GitHub Copilot's 55.8% faster study, Google's approximately 21% enterprise RCT estimate, and the 30.7% median speedup in the preregistered feature-implementation study all measure short-horizon completion on defined tasks rather than full software delivery pipelines.[35] [10] [69]

Enterprise contexts sit in the middle. They can still show meaningful gains because work is often standardized, internal tooling can be optimized, and organizations can target AI toward

repetitive implementation tasks. However, enterprise productivity is also filtered through review queues, security checks, testing obligations, and release governance, which is why even positive enterprise findings come with caution about generalization and why mixed-method enterprise studies argue for broader metrics such as ownership, expertise, and long-term maintainability rather than raw speed alone.[10] [4]

Open-source contexts are the least forgiving because they combine mature codebases, heterogeneous contributors, public quality scrutiny, and a high premium on maintainability. In the METR RCT, experienced maintainers working on repositories they knew well were slower with AI, and in the OSS maintenance-burden study, gains accrued mainly to peripheral contributors while core developers absorbed more review and rework. These findings suggest that open-source work exposes the hidden cost of AI more clearly than lab tasks do because the work is dominated by context reconstruction and quality control rather than by blank-page generation.[2] [73]

The longitudinal evidence helps explain why these contexts diverge operationally. Over six months, developers reported a shift from creation toward verification and correction, which the authors describe as supervisory engineering work. That shift is manageable in bounded experiments, partially absorbable in enterprise teams with process support, and especially costly in open-source environments where expert maintainers are scarce and review capacity is limited.[3]

For decision-makers, the context lesson is more important than any single headline number.

- Laboratory context: Best for demonstrating technical potential, but prone to overstating real-world ROI because it suppresses coordination and maintenance costs.[35] [1]
- Enterprise context: Most relevant for procurement, but effects depend on workflow design, review capacity, and whether the organization measures shipped outcomes rather than coding activity alone.[10] [4]
- Open-source context: Most revealing about expert maintenance burden and verification overhead, making it a useful stress test for claims of universal productivity gain.[2] [73]

The measured-versus-claimed conclusion for this section is therefore narrow but robust. AI coding assistants do not have a single productivity effect that travels unchanged across populations and settings; they produce larger gains for less experienced developers on bounded generation-heavy tasks, and smaller or negative gains for experienced developers doing high-context maintenance, review, and integration work, especially in open-source-style environments where verification costs are hardest to hide.[1] [2] [73]

9

Code Quality and Maintainability Evidence

9

Code Quality and Maintainability Evidence

The maintainability evidence is materially thinner than the short-run productivity evidence, but it is also more cautionary. Across the sources available as of August 25, 2026, the strongest measured pattern is not a proven collapse in code quality, but a growing signal that AI-assisted development can increase rework, duplication, and review burden even when it accelerates initial code production.[173] [73] [3]

At the same time, the best controlled maintainability experiment in this report does not find measurable downstream harm or benefit when later developers evolve AI-assisted code under the study's conditions. The most defensible synthesis is therefore narrow: current evidence supports concern about maintenance burden and code churn proxies, but it does not yet justify a universal claim that AI assistance systematically damages long-term codebase health in every setting.[156] [73] [173]

9.1

GitClear analysis: code churn and revert rates doubling in AI-assisted codebases

GitClear's code quality research is one of the most cited large-scale observational signals in this debate because it looks beyond speed and volume to structural indicators associated with maintainability. In the 2025 report covering 2020 to 2024 repository history, GitClear argues that AI-era coding is associated with less reuse and refactoring, more duplication, and materially higher short-horizon churn, which it treats as a proxy for defect introduction or rework.[173]

Title — GitClear maintainability signals

SIGNAL	REPORTED CHANGE	MAINTAINABILITY IMPLICATION
Two-week code churn	3.1% in 2020 to 5.7% in 2024	More recently authored code is being revised or removed quickly.
Refactoring or moved code	Less than half the 2020 rate by 2024	Less structural reuse and consolidation work.
Duplicate code blocks	8x increase across 2020 to 2024	Greater risk of parallel logic and fragmented fixes.
Legacy maintenance	74% decline	Less effort appears directed to upkeep of existing systems.

Source: [173]

- Why the findings matter: These are not direct defect counts, but they are maintainability-relevant proxies. A codebase with more duplication, less code movement or reuse, and more rapid post-commit revision is usually harder to reason about and more expensive to evolve over time.[173]
- Why caution is required: GitClear is an observational analysis produced by a company whose product is built around code quality measurement, so its results should be treated as strong warning signals rather than definitive causal proof. The report shows temporal association with

the AI era, but it cannot fully isolate AI from other concurrent changes in team process, repository mix, or coding norms.[173]

- Why the signal is still hard to dismiss: Independent open-source evidence points in the same direction. A 2025 OSS study finds that code written after Copilot adoption requires more rework, while experienced core developers review 6.5% more code and suffer a 19% drop in their own original-code productivity, suggesting that at least part of the apparent speed gain is being converted into downstream maintenance labor.[73]

For this report's measured-versus-claimed framing, GitClear is important because it targets the part of the workflow that vendor productivity claims usually omit. Faster code generation can coexist with worse maintainability signals, and GitClear's contribution is to show that this is not merely a theoretical concern but an empirically observable pattern in large repository histories, even if the exact causal share attributable to AI remains uncertain.[173] [73]

9.2

Two-phase RCT Phase 2: no measurable harm or benefit to code evolution time or quality

The strongest counterweight to the GitClear-style caution comes from the two-phase preregistered RCT discussed earlier in this report. Its Phase 2 design is unusually valuable because it tests whether code initially written with AI becomes easier or harder for a different developer to evolve later, which is much closer to a maintainability question than simple first-author speed.[156]

- What Phase 2 tested: A second set of participants modified codebases produced in Phase 1 without AI assistance, allowing the researchers to compare downstream evolution effort rather than just initial implementation speed. Under those study conditions, they found no significant difference in code evolution time or code quality between code originally produced with AI assistance and code produced without it.[156]
- Why this matters: It is one of the few studies in the literature that directly examines whether AI leaves a measurable maintainability signature for the next developer. That makes it more decision-relevant than studies that stop at task completion or code volume.[156]
- What the null result means: The study does not prove that AI is maintainability-neutral in general. It shows that, for the specific Java web application tasks, participant pool, and quality measures used, any downstream effect was too small or too inconsistent to detect reliably.[156]

This neutral finding is analytically important because it prevents overcorrection. The current evidence does not support a simple story in which AI-assisted code is always worse for future maintainers; rather, it suggests that maintainability effects may depend heavily on task boundedness, repository complexity, and how much of the work is generation versus integration.[156] [73]

A second 2026 empirical study on agent-generated pull requests also complicates the picture. Using over 1,000 files and about 3,200 changes from 100 popular repositories, it finds that AI-generated files receive less frequent maintenance than human-authored code and that the most common later modifications are feature extensions rather than bug fixes, with humans performing most of the maintenance work; this does not establish superior quality, but it does show that not all AI-authored artifacts immediately degrade into high-churn liabilities.[156]

9.3

What the evidence does and does not say about long-term codebase health

What the evidence does say is relatively specific. AI assistance can increase the amount of code entering the system, shift developer effort from creation toward verification, and in some settings raise rework and review burden, especially for experienced maintainers who must absorb the downstream consequences of faster upstream generation.[3] [73] [173]

- What is reasonably supported: Maintainability risk is real enough to monitor. The convergence between GitClear's churn and duplication signals, the OSS maintenance-burden study's rework findings, and the longitudinal study's shift toward supervisory engineering work all point to a plausible mechanism in which local drafting gains create downstream validation and upkeep costs.[173] [73] [3]
- What is not yet supported: There is still no strong causal evidence that AI assistance universally worsens long-run defect density, architectural coherence, or total cost of ownership across mature production systems. Most studies remain short-horizon, proxy-based, or context-specific, and the best controlled maintainability experiment so far found no measurable downstream harm.[156]
- What remains unknown: The field still lacks multi-year causal studies linking AI usage intensity to release stability, incident rates, security defects, architectural drift, and maintenance cost at the codebase level. That gap is especially important because maintainability problems often emerge slowly, after repeated small compromises accumulate.[173] [3]

The most balanced conclusion for industry readers is therefore conditional rather than binary. Current measured evidence is sufficient to reject the claim that faster code generation is automatically maintainability-neutral, but it is not sufficient to conclude that AI-assisted coding inevitably harms long-term codebase health in every environment.[73] [156]

For decision-makers, the practical implication is to treat maintainability as a first-class evaluation dimension during rollout. Organizations should measure churn, revert rates, duplication, review load, post-merge fixes, and expert rework alongside speed metrics, because the current literature suggests that the main risk is not necessarily that AI writes unusable code, but that it can quietly shift the economics of codebase upkeep in ways that short-run productivity dashboards fail to capture.[173] [3] [73]

10

Conclusions and Practical Implications

10

Conclusions and Practical Implications

Across the evidence base reviewed in this report, the most defensible conclusion is that AI coding assistants are neither a universal productivity breakthrough nor a negligible aid. As of August 25, 2026, the weight of independent evidence supports a moderate average productivity benefit under some conditions, but with large variation by task type, developer experience, and work context, and with materially weaker support for broad vendor-style claims of large, generalizable speedups.[1] [10]

For organizations, the practical implication is that evaluation should move away from headline percentages and toward local measurement of end-to-end delivery outcomes, review burden, maintainability, and skill effects. The current literature is strong enough to justify disciplined experimentation, but not strong enough to justify assuming that visible coding acceleration will automatically translate into proportional business value, durable code quality, or preserved developer capability.[3] [4]

10.1

What the weight of evidence supports, and what remains genuinely uncertain

The strongest report-wide conclusion is conditional rather than absolute. The May 2026 meta-analysis found a statistically significant moderate productivity effect of Hedges' g = 0.33 with 95% CI [0.09, 0.58], but also reported substantial heterogeneity and explicitly noted that gains are larger in controlled settings and smaller in open-source and enterprise contexts.[1]

- Supported conclusion: AI coding assistants can improve short-run productivity on bounded, generation-heavy tasks where correctness is locally checkable and first-draft production is a large share of the work.[1] [10]
- Supported conclusion: Measured gains are usually smaller and less portable than vendor claims. Google's enterprise RCT estimated about a 21% reduction in time on task with wide confidence intervals, which is directionally positive but materially below the most aggressive marketing figures and explicitly not assumed to generalize across tools or time periods.[10]
- Supported conclusion: Perceived productivity is not a reliable proxy for measured productivity. The six-month longitudinal study found 84% stable perceived productivity improvement while developer experience worsened on some dimensions, and the broader literature reviewed in this report repeatedly shows that reduced writing effort can coexist with increased verification burden.[3] [4]
- Supported conclusion: Productivity effects are moderated most strongly by context and experience. The meta-analysis, longitudinal evidence, and mixed-method enterprise evidence all point to the same pattern that local coding acceleration does not map cleanly onto system-level productivity in mature delivery environments.[1] [3] [4]
- Supported conclusion: The evidence for learning and capability benefits is weak. The same May 2026 meta-analysis found no statistically significant overall learning effect, with Hedges' g = 0.14 and 95% CI [-0.18, 0.47], which is consistent with this report's broader caution that short-run assistance should not be assumed to strengthen long-run skill development.[1]

What remains uncertain is equally important.

- Genuine uncertainty: The true long-run effect on codebase health remains unresolved. Current studies provide warning signals and some neutral findings, but the field still lacks multi-year causal evidence linking AI usage intensity to defect escape, architectural drift, incident rates, or total maintenance cost.[1] [3]
- Genuine uncertainty: The net effect on senior developers and high-context maintenance work is still unstable across settings. Existing evidence suggests that verification and integration overhead can offset drafting gains, but the exact boundary conditions are not yet well quantified across industries and repository types.[1] [4]
- Genuine uncertainty: Enterprise ROI at the organizational level is not yet well established. The literature is much stronger on task-level speed and activity metrics than on release throughput, customer outcomes, or cost-adjusted business value.[10] [4]
- Genuine uncertainty: The field still lacks a stable shared definition of productivity. Because studies use different proxies and time horizons, some disagreement in results reflects measurement fragmentation rather than direct contradiction.[4] [105]

10.2

Recommendations for organisations evaluating AI coding tools objectively

Organizations should evaluate AI coding tools as socio-technical workflow interventions, not as standalone code generators. The evidence reviewed in this report supports disciplined local experimentation, but it does not support procurement decisions based primarily on vendor headline percentages or employee enthusiasm alone.[1] [4]

- Use representative pilots: Test tools on real internal workflows that include implementation, review, debugging, integration, and release steps, rather than on isolated coding demos. Controlled studies tend to overstate realized enterprise gains because they suppress downstream friction.[1] [10]
- Segment by developer cohort: Evaluate junior, mid-level, and senior developers separately. The literature indicates that benefits are often concentrated in less experienced developers, while senior staff may absorb more review, correction, and architectural burden.[1] [3]
- Segment by task type: Measure outcomes separately for boilerplate, bounded feature work, debugging, review, and design. The current evidence base does not support a single average effect across these categories.[105] [1]
- Prefer end-to-end metrics over activity proxies: Track cycle time, review latency, merge success, post-merge fixes, release throughput, and rework, not just lines of code, commits, or accepted suggestions. The literature repeatedly warns that upstream activity gains can overstate delivered value.[4] [3]
- Measure maintainability explicitly: Add indicators such as churn, duplication, revert rates, review burden, and time-to-modify by a second developer. Current evidence is too mixed to assume maintainability neutrality.[1] [4]
- Include human factors in the scorecard: Track flow state, cognitive load, confidence calibration, and perceived ownership of work alongside throughput. The six-month longitudinal study shows that stable perceived gains can coexist with worsening developer experience.[3]
- Protect learning pathways: For junior developers especially, preserve some unaided work, explanation-based review, and deliberate debugging practice. The evidence base does not support assuming that faster completion is skill-neutral over time.[1] [105]

- Treat vendor claims as hypotheses: Use them to define test cases, not expected ROI. A claim derived from a bounded task study or self-report should be validated against the organization's own repositories, governance model, and staffing mix before scaling.[10] [4]

A practical evaluation model follows directly from the evidence.

- Before rollout: Define success metrics that include speed, quality, maintainability, and capability development.[4]
- During pilot: Randomize where feasible, or at minimum use matched teams and pre-post baselines with task-level segmentation.[10] [1]
- After pilot: Scale only where measured gains survive review and release stages, and where no material deterioration appears in maintainability or developer capability indicators.[3] [4]

10.3

Key gaps in the research and what future studies should address

The current literature is already large enough to reject simplistic claims, but it is still too fragmented to support precise, universal operational benchmarks. Future research should shift from asking whether AI helps in general to identifying when, for whom, and at what downstream cost it helps.[1] [105]

- Gap: Long-run causal evidence is scarce. Most studies remain short-horizon, so future work should track teams for 12 to 24 months and link AI usage to release stability, defect escape, incident rates, architectural drift, and maintenance cost.[1] [3]
- Gap: Team-level and organizational outcomes are undermeasured. Future studies should examine whether local coding gains survive review queues, testing bottlenecks, security checks, and release governance, rather than stopping at task completion or code volume.[10] [4]
- Gap: Measurement remains fragmented. The field needs a smaller common core of validated productivity metrics spanning throughput, quality, maintainability, and human factors so that results can be compared across studies and organizations.[4] [105]
- Gap: Seniority effects need sharper causal identification. Future trials should be powered to estimate heterogeneous treatment effects for junior, mid-level, and senior developers separately, because current evidence suggests materially different benefit and burden profiles.[1] [3]
- Gap: Task taxonomy is still too coarse. Studies should distinguish greenfield implementation, maintenance, debugging, review, refactoring, architecture, and incident response, since these are not interchangeable productivity domains.[105] [1]
- Gap: Learning and skill retention evidence is thin relative to productivity evidence. Future studies should include repeated knowledge assessments, reasoning quality measures, and promotion-readiness indicators, especially for early-career developers.[1] [105]
- Gap: Comparative tool studies are limited. Much of the literature evaluates one tool or one internal stack at a time, so future work should compare multiple assistant types and interaction modes under the same task and governance conditions.[10] [1]
- Gap: Human oversight costs are not yet measured with enough precision. Future studies should quantify prompt iteration time, verification time, review spillovers, and expert rework so that the full economics of supervisory engineering work can be estimated directly.[3]

The practical research agenda is therefore clear. The next generation of studies should be longitudinal, preregistered where feasible, powered for subgroup analysis, and designed around end-to-end software delivery rather than isolated code generation tasks.[1] [10] [105]

11

References and Citations

11

References and Citations

This section consolidates the source base used across the report and separates empirical primary evidence from contextual secondary material. The emphasis is on traceable, study-specific citations that distinguish independently measured outcomes from vendor communications, industry analyses, and other interpretive or non-peer-reviewed sources.

11.1

Primary Sources

The report's empirical backbone is built on controlled experiments, quasi-experiments, meta-analyses, systematic reviews, and large-scale observational studies that directly measured productivity, maintainability, learning, or workflow effects of AI coding assistants.

- Peng, Sida, Eirini Kalliamvakou, Peter Cihon, and Mert Demirer. "The Impact of AI on Developer Productivity: Evidence from GitHub Copilot." arXiv, February 13, 2023.[35]
- Paradis, Elise, Kate Grey, Quinn Madison, Daye Nam, Andrew Macvean, Vahid Meimand, Nan Zhang, and Ben Ferrari-Church. "How much does AI impact development speed? An enterprise-based randomized controlled trial." arXiv, October 16, 2024.[10]
- Becker, Joel, Nate Rush, Elizabeth Barnes, and David Rein. "Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity." arXiv, July 12, 2025.[2]
- Borg, Markus, Dave Hewett, Nadim Hagatullah, Noric Couderc, Emma Söderberg, Donald Graham, Uttam Kini, and Dave Farley. "Echoes of AI: Investigating the Downstream Effects of AI Assistants on Software Maintainability." arXiv, July 1, 2025.[72]
- Borg, Markus, Dave Hewett, Donald Graham, Noric Couderc, Emma Söderberg, Luke Church, and Dave Farley. "Does Co-Development with AI Assistants Lead to More Maintainable Code? A Registered Report." arXiv, August 20, 2024.[69]
- Maier, Sebastian, Moritz Gunzenhäuser, Jonas Schweisthal, Manuel Schneider, and Stefan Feuerriegel. "A meta-analysis of the effect of generative AI on productivity and learning in programming." arXiv, May 6, 2026.[1]
- Stray, Viktoria, Elias Goldmann Brandtzæg, Viggo Tellefsen Wivestad, Astri Barbala, and Nils Brede Moe. "Developer Productivity With and Without GitHub Copilot: A Longitudinal Mixed-Methods Case Study." arXiv, September 24, 2025.[184]
- Chen, Valerie, Jasmyn He, Benjamin Williams, Jason Valentino, and Ameet Talwalkar. "Beyond the Commit: Developer Perspectives on Productivity with AI Coding Assistants." arXiv, February 3, 2026.[4]
- Xu, Feiyang, Poonacha K. Medappa, Murat M. Tunc, Martijn Vroegindeweij, and Jan C. Fransoo. "AI-assisted Programming May Decrease the Productivity of Experienced Developers by Increasing Maintenance Burden." arXiv, October 11, 2025.[73]
- Noy, Shakked, and Whitney Zhang. "Writing Code vs. Shipping Code: Productivity Effects Across Generations of AI Coding Tools." NBER Working Paper No. 35275, 2026.[90]
- Anthropic. "How AI assistance impacts the formation of coding skills." January 29, 2026.[135]

These sources were prioritized because they provide direct measurement, explicit methods, and study-level results rather than generalized marketing summaries. Where a study existed both as a registered report and as a results paper, both were retained to preserve methodological traceability and preregistration context.[1] [10] [2]

11.2

Secondary Sources

Secondary sources were used for comparison, contextualization, and interpretation, especially where vendor claims, industry telemetry, public-sector trials, or code-quality warning signals were relevant but did not meet the same evidentiary standard as randomized or peer-oriented primary studies.

- GitHub Research. “Research: quantifying GitHub Copilot’s impact on developer productivity and happiness.” 2022.[50]
- OpenAI. “OpenAI Productivity Note.” July 2025.[62]
- OpenAI. “The State of Enterprise AI 2025 Report.” 2025.[58]
- GitClear. “AI Copilot Code Quality: Evaluating 2024’s Increased Defect Rate with Data.” 2025.[173]
- GitClear. “AI Code Quality Signal Graphs.” 2026 access point for longitudinal quality indicators.[166]
- Additional contextual sources cited in earlier sections but not treated as primary causal evidence include conference abstract pages and organizational research summaries where full peer-reviewed manuscripts were not directly available through the search process. These were used sparingly and only to support claims about the existence, framing, or public reporting of studies rather than to substitute for primary empirical evidence.[135] [90]

The distinction applied throughout the report is therefore evidentiary rather than topical. Primary sources were used to support measured outcomes, while secondary sources were used to document vendor claims, public narratives, implementation context, and non-peer-reviewed warning signals that informed comparison but were not treated as definitive proof.[35] [62] [173]

Caspr.

ANALYTICAL AI — BUILT FOR BUSINESS ANALYSIS, NOT CONVERSATION.

ABOUT CASPR.

Caspr is **Analytical AI** — purpose-built for business analysis, not conversation. A single prompt delivers a boardroom-ready report in under fifteen minutes, citing every insight to a credible, verifiable source.

Powered by **the Thinking Brain** — a rigorous research methodology that reasons through data and structures defensible conclusions — and **the Learning Brain**, which draws on real-time data feeds to ensure analysis reflects today's reality. Over one million curated sources: government databases, industry filings, news feeds, and published research. Not web-scraped noise.

Every claim cited. Every conclusion traceable. Built for the boardroom.

CONTACT

WEB caspr.ai

EMAIL hello@caspr.ai

SUPPORT support@caspr.ai

DISCLAIMER

This report has been prepared by Caspr for informational purposes only. The analysis, data, and conclusions contained herein are based on sources believed to be reliable at the time of preparation. Caspr makes no representation or warranty, express or implied, as to the accuracy, completeness, or fitness for purpose of any information in this report.

This report does not constitute investment, legal, financial, or professional advice of any kind. Recipients should seek independent professional advice before acting on any information or analysis contained in this report.

Caspr accepts no liability for any direct, indirect, or consequential loss or damage arising from the use of this report or reliance on its contents. The views and analysis expressed are those derived from Caspr's research methodology and do not represent the views of any third party whose data has been referenced.

All third-party source material is referenced for informational purposes. Reproduction of this report in whole or in part requires prior written permission from Caspr.

METHODOLOGY

Analysis is generated using Caspr's Thinking Brain methodology, drawing on a curated database of over one million verified sources. Every factual claim in this report is cited to a primary source. Where calculations or derivations are Caspr's own, this is indicated as "Caspr analysis" alongside the underlying source data.